```
// EXP: 24
// NAME: C NEETHU NAVEENA
// ROLL NO: 192311025
// CLASS: CSE
// ECA1408 EMBEDDED SYSTEM
#include <systemc.h>

SC_MODULE(ClockExample) {
sc_in<bool> clk;

void process() {
while (true) {
wait();
std::cout << "Time: " << sc_time_stamp()
<< " - Clock Triggered" << std::endl;
}
}

SC_CTOR(ClockExample) {
SC_THREAD(process);
sensitive << clk.pos();

}
};

int sc_main(int argc, char* argv[]) {

sc_clock clock("clock", 1, SC_NS);

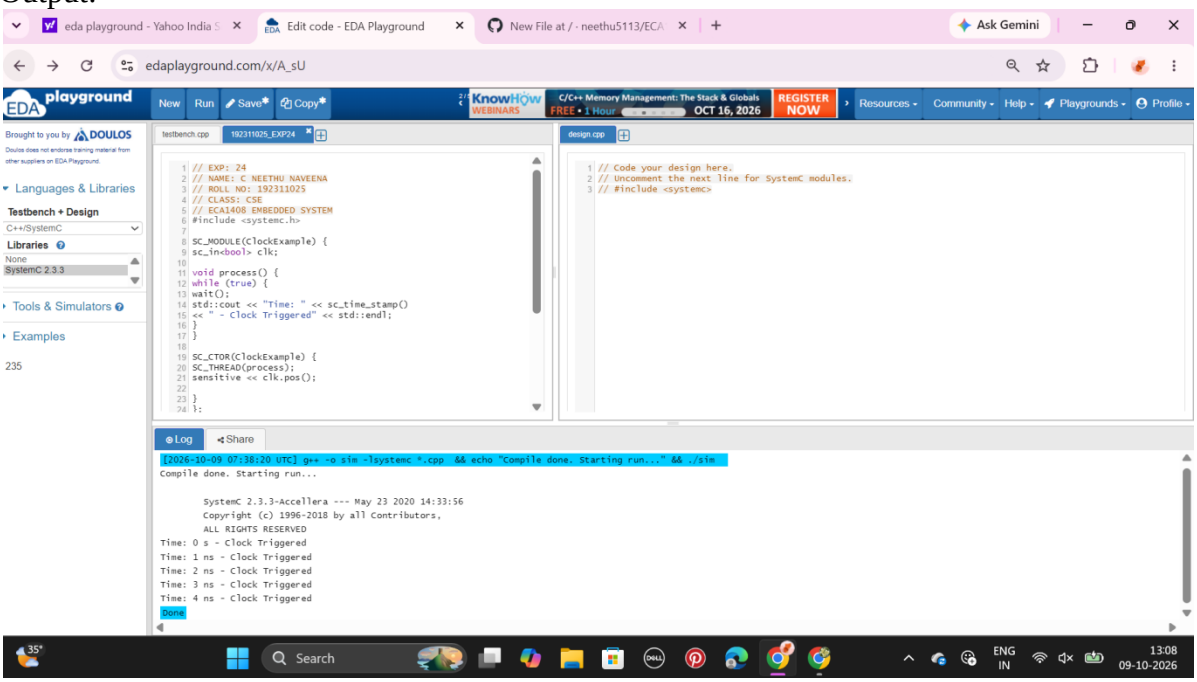
ClockExample module("module");
module.clk(clock);
```

```
sc_start(5, SC_NS);


return 0;
}
```

Result: Thus, the clock-driven process using SC_THREAD and wait() was successfully implemented and verified in SystemC.

Output:




```
// EXP: 25
// NAME: C NEETHU NAVEENA
// ROLL NO: 192311025
// CLASS: CSE
// ECA1408 EMBEDDED SYSTEM
#include <systemc.h>
// Producer Module
SC_MODULE(Producer) {
sc_fifo_out<int> out;
void produce() {
for (int i = 1; i <= 5; i++) {
```

```
cout << "Produced: " << i << endl;
out.write(i);
wait(1, SC_SEC);
}
}
SC_CTOR(Producer) {
SC_THREAD(produce);
}
};
// Consumer Module
SC_MODULE(Consumer) {
sc_fifo_in<int> in;
void consume() {
int value;
while (true) {
in.read(value);
cout << "Consumed: " << value << endl;
wait(1, SC_SEC);
}
}
SC_CTOR(Consumer) {
SC_THREAD(consume);
}
};
int sc_main(int argc, char* argv[]) {
sc_fifo<int> fifo(5);
Producer producer("Producer");
Consumer consumer("Consumer");

producer.out(fifo);
consumer.in(fifo);
```

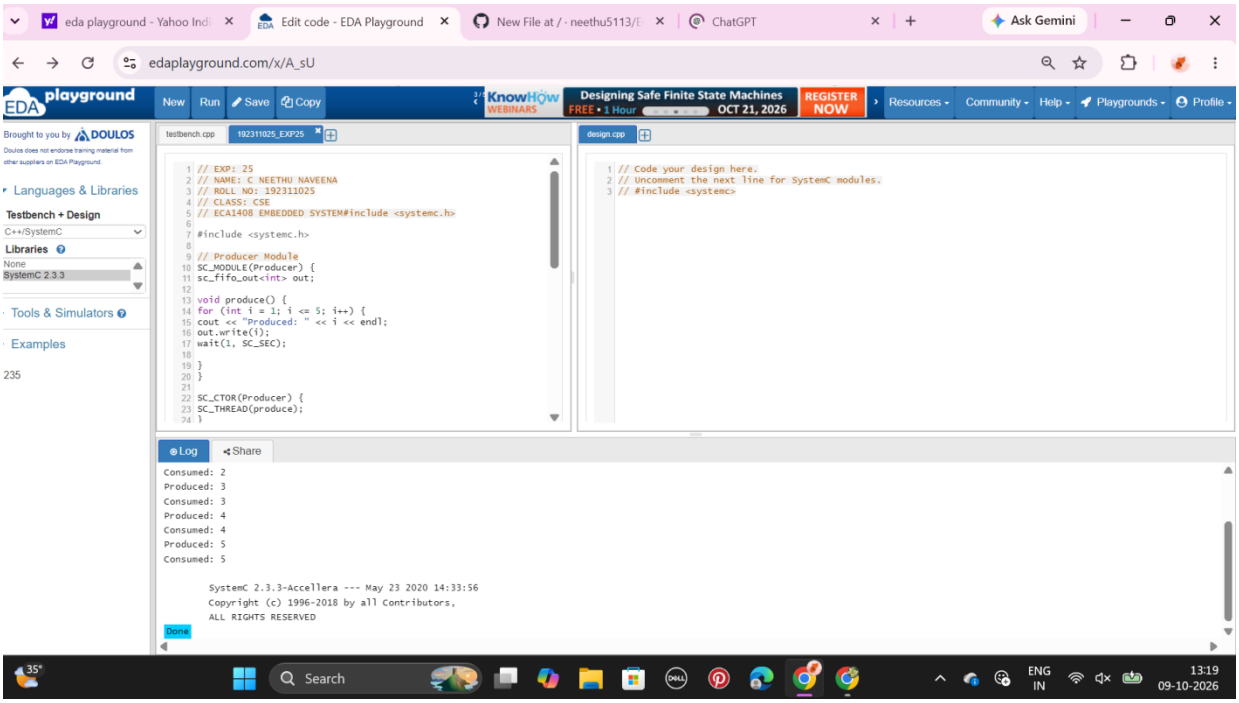
sc_start(10, SC_SEC);

return 0;

}

RESULT: Thus, the Producer–Consumer problem was successfully implemented and simulated using SystemC. Proper synchronization between concurrent processes was achieved using FIFO communication.

OUTPUT:



// EXP: 26

// NAME: C NEETHU NAVEENA

// ROLL NO: 192311025

// CLASS: CSE

// ECA1408 EMBEDDED SYSTEM

#include <systemc>

#include <tlm>

#include <tlm_utils/simple_initiator_socket.h>

```
#include <tlm_utils/simple_target_socket.h>

using namespace sc_core;
using namespace tlm;
using namespace std;

// Target
struct Target : sc_module {
tlm_utils::simple_target_socket<Target> socket;

SC_CTOR(Target) : socket("socket") {
socket.register_b_transport(this, &Target::b_transport);
}

void b_transport(tlm_generic_payload& trans, sc_time& delay) {
int* data = (int*) trans.get_data_ptr();
cout << "Target received: " << *data << endl;

delay += sc_time(10, SC_NS);
}
};

// Initiator
struct Initiator : sc_module {
tlm_utils::simple_initiator_socket<Initiator> socket;
```

```
SC_CTOR(Initiator) : socket("socket") {
SC_THREAD(thread_process);
}

void thread_process() {
tlm_generic_payload trans;
sc_time delay = SC_ZERO_TIME;

int data = 555;

trans.set_data_ptr(reinterpret_cast<unsigned char*>(&data));
trans.set_data_length(sizeof(data));

cout << "Initiator sending: " << data << endl;
socket->b_transport(trans, delay);

wait(delay);
}
};

int sc_main(int argc, char* argv[]) {

Initiator initiator("initiator");
Target target("target");
```

initiator.socket.bind(target.socket);

sc_start();

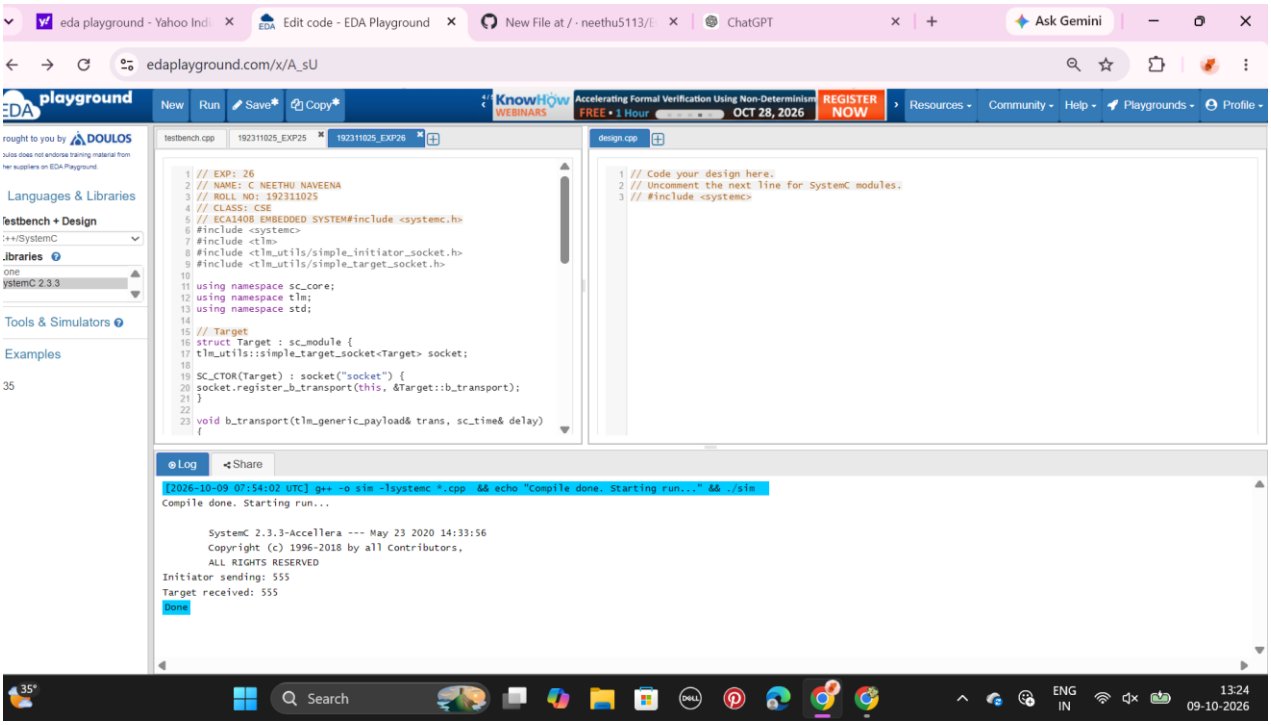
return 0;

}

RESULT:

Thus the Transaction-Level Model using Initiator and Target modules was successfully implemented and simulated using SystemC.

OUTPUT:



```
// EXP: 27
// NAME: C NEETHU NAVEENA
// ROLL NO: 192311025
// CLASS: CSE
// ECA1408 EMBEDDED SYSTEM
#include <systemc.h>

SC_MODULE(binary_semaphore) {

int sem;
sc_event ev;

void task1() {
while(true) {
wait(1, SC_SEC);

if(sem == 1) {

sem = 0;
cout << "Task1 Entered Critical Section at " << sc_time_stamp() << endl;
wait(2, SC_SEC);
sem = 1;
ev.notify();
}
else {
wait(ev);
}
}
}
```

```
void task2() {
while(true) {
wait(2, SC_SEC);

if(sem == 1) {
sem = 0;
cout << "Task2 Entered Critical Section at " << sc_time_stamp() << endl;
wait(1, SC_SEC);
sem = 1;
ev.notify();
}
else {
wait(ev);
}
}
}

SC_CTOR(binary_semaphore) {
sem = 1;
SC_THREAD(task1);
SC_THREAD(task2);
}

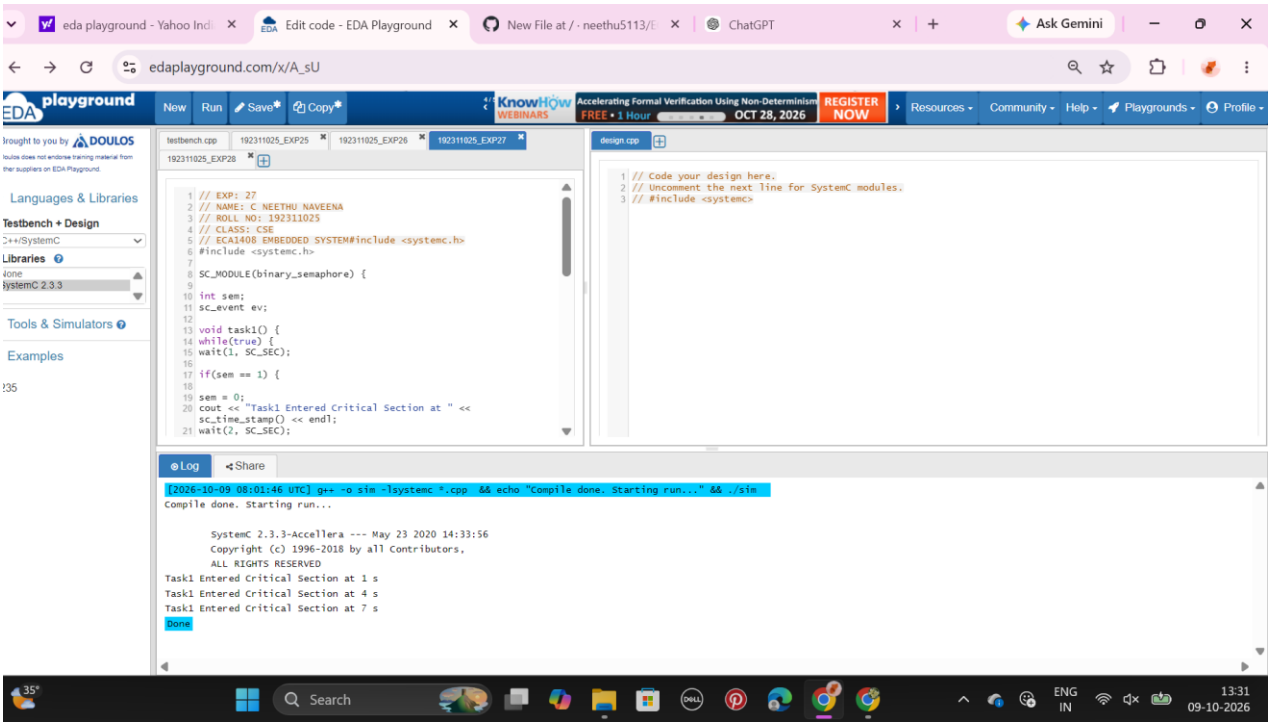
};

int sc_main(int argc, char* argv[]) {
binary_semaphore obj("Binary_Semaphore");
sc_start(10, SC_SEC);
return 0;
}
```

RESULT:

Thus the Binary Semaphore was created using SystemC and verified.

OUTPUT:



// EXP: 28

// NAME: C NEETHU NAVEENA

// ROLL NO: 192311025

// CLASS: CSE

// ECA1408 EMBEDDED SYSTEM

#include <systemc.h>

SC_MODULE(counting_semaphore) {

int sem;

void task() {

while(true) {

wait(1, SC_SEC);

```
if(sem > 0) {
sem--;
cout << "Task Accessing Resource at "
<< sc_time_stamp()
<< " Remaining: " << sem << endl;

wait(2, SC_SEC);
sem++;
}

}
}

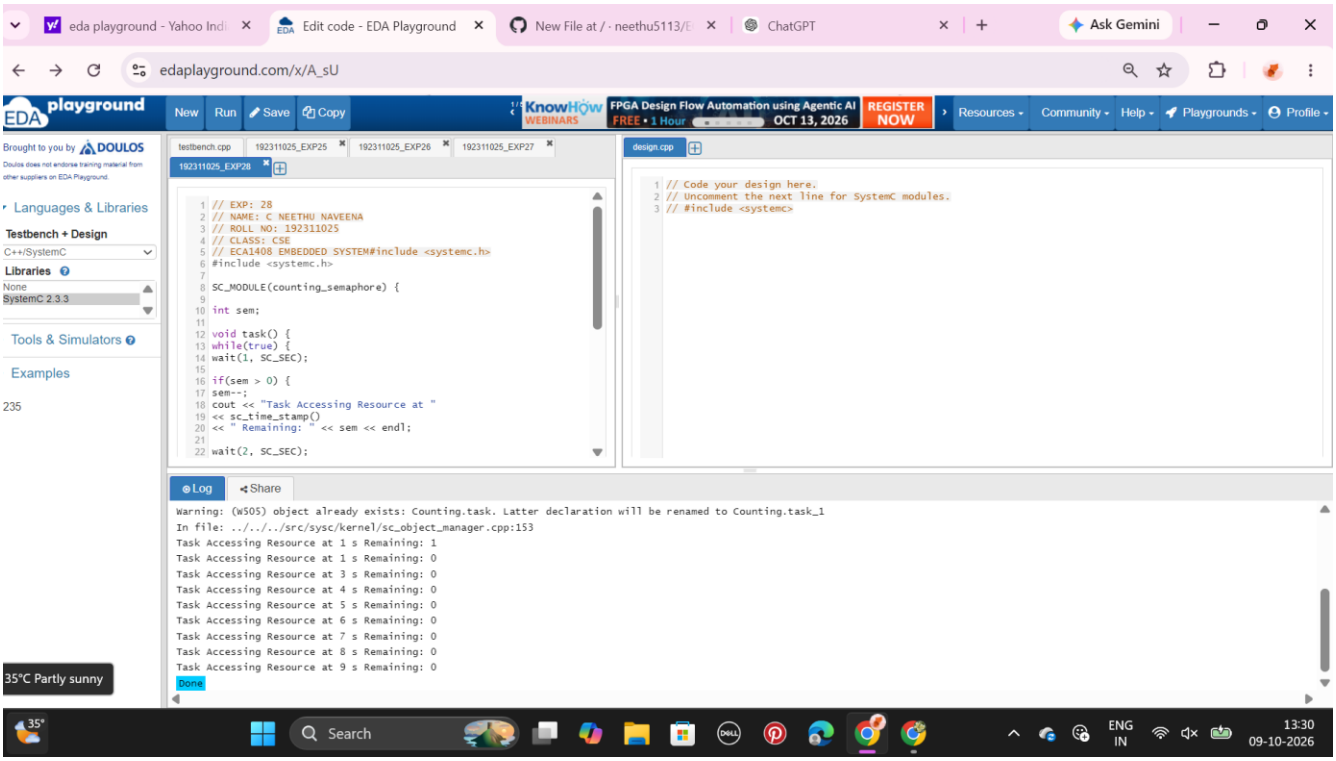
SC_CTOR(counting_semaphore) {
sem = 2;
SC_THREAD(task);
SC_THREAD(task);
SC_THREAD(task);
}
};

int sc_main(int argc, char* argv[]) {
counting_semaphore obj("Counting");
sc_start(10, SC_SEC);
return 0;
}
```

RESULT:

Thus the Counting Semaphore was created using SystemC and verified

OUTPUT:



// EXP: 29

// NAME: C NEETHU NAVEENA

// ROLL NO: 192311025

// CLASS: CSE

// ECA1408 EMBEDDED SYSTEM

#include <systemc.h>

SC_MODULE(priority_interrupt) {

sc_event low_int, high_int;

void low_ISR() {

while(true) {

```
wait(low_int);
cout << "Low Priority Interrupt at "
<< sc_time_stamp() << endl;
wait(3, SC_SEC);
}
}

void high_ISR() {
while(true) {
wait(high_int);
cout << "High Priority Interrupt at "

<< sc_time_stamp() << endl;
wait(1, SC_SEC);
}
}

void generator() {
wait(1, SC_SEC);
low_int.notify();

wait(1, SC_SEC);
high_int.notify();
}

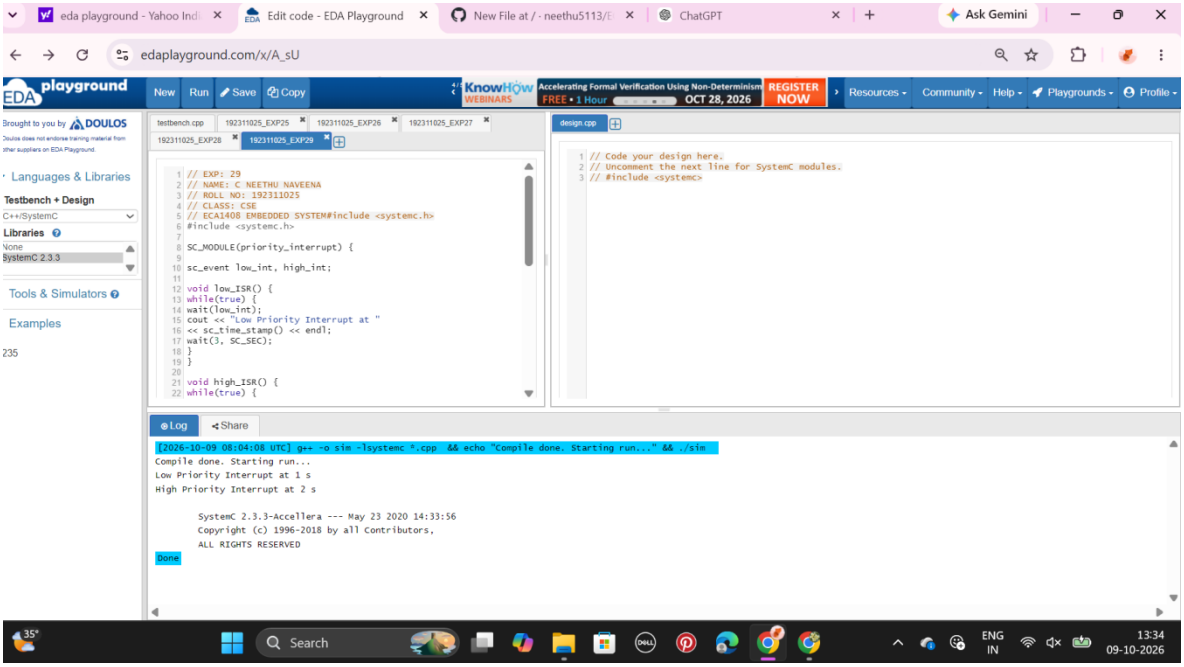
SC_CTOR(priority_interrupt) {
SC_THREAD(low_ISR);
SC_THREAD(high_ISR);
SC_THREAD(generator);
}
};
```

```
int sc_main(int argc, char* argv[]) {
priority_interrupt obj("Priority_INT");
sc_start(10, SC_SEC);
return 0;
}
```

RESULT:

Thus the Priority based Interrupt handling was executed using SystemC and verified.

OUTPUT:




```
// EXP: 30
// NAME: C NEETHU NAVEENA
// ROLL NO: 192311025
// CLASS: CSE
// ECA1408 EMBEDDED SYSTEM
#include <systemc.h>
SC_MODULE(rtos_scheduler) {
void task1() {
while(true) {
cout << "Task1 Running at "
```

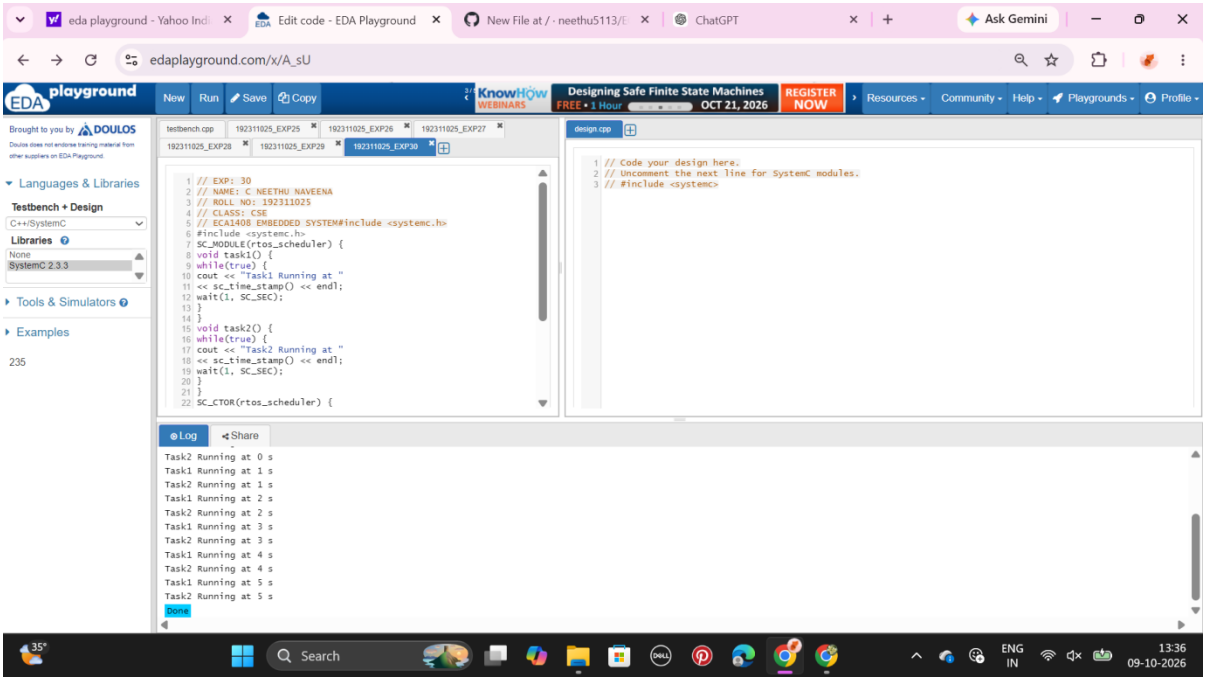
```
<< sc_time_stamp() << endl;
wait(1, SC_SEC);
}
}
void task2() {
while(true) {
cout << "Task2 Running at "
<< sc_time_stamp() << endl;
wait(1, SC_SEC);
}
}
SC_CTOR(rtos_scheduler) {
SC_THREAD(task1);
SC_THREAD(task2);
}
};

int sc_main(int argc, char* argv[]) {
rtos_scheduler obj("Scheduler");
sc_start(6, SC_SEC);
return 0;
}
```

RESULT:

Thus the Round-Robin scheduling was executed using SystemC and verified.

OUTPUT:


edaplayground.com/x/A_sU
playground
New
Run
Save
Copy
Designing Safe Finite State Machines
FREE • 1 Hour
OCT 21, 2026
REGISTER NOW
Resources
Community
Help
Playgrounds
Profile
Languages & Libraries
Testbench + Design
C++/SystemC
Libraries
SystemC 2.3.3
Tools & Simulators
Examples
235
testbench.cpp
192311025_EXP25
192311025_EXP26
192311025_EXP27
192311025_EXP28
192311025_EXP29
192311025_EXP30
design.cpp
// EXP: 30
// NAME: C NEETHU NAVEENA
// ROLL NO: 192311025
// CLASS: CSE
// ECA1408 EMBEDDED SYSTEM#include <systemc.h>
#include <systemc.h>
SC_MODULE(rtos_scheduler) {
void task1() {
while(true) {
cout << "Task1 Running at "
<< sc_time_stamp() << endl;
wait(1, SC_SEC);
}
}
void task2() {
while(true) {
cout << "Task2 Running at "
<< sc_time_stamp() << endl;
wait(1, SC_SEC);
}
}
SC_CTOR(rtos_scheduler) {
// Code your design here.
// Uncomment the next line for SystemC modules.
// #include <systemc>
Log
Share
Task2 Running at 0 s
Task1 Running at 1 s
Task2 Running at 1 s
Task1 Running at 2 s
Task2 Running at 2 s
Task1 Running at 3 s
Task2 Running at 3 s
Task1 Running at 4 s
Task2 Running at 4 s
Task1 Running at 5 s
Task2 Running at 5 s
Done
Search
ENG
IN
13:36
09-10-2026